

Measurement-Based Swarm Optimisation with Controlled Decoherence
for Re-Scheduling Cloud Workloads under Change:
a Quantum-Inspired, Classically Executed Approach to Cloud Resource Management

PhD Research Proposal

by

Ayush Prajapati

B.Tech Computer Engineering (CPI 8.84/10), PDEU Gandhinagar, 2022-2026
DevOps Engineer (AWS · Terraform · Docker · Kubernetes)

1. The rationale for doing the research

The major objectives of my research are the following:

- to build and contribute to the knowledge of population-based (swarm) optimisation for cloud task scheduling and resource management, in particular to establish *which representation of a schedule* a swarm optimiser should search over, and why the representation used by most published cloud-scheduling metaheuristics fails as the problem grows;
- to develop and rigorously evaluate a measurement-based schedule representation — inspired by quantum measurement theory (superposition, Born-rule measurement, decoherence) but executed entirely on classical CPUs — that gives swarm optimisers a controllable move size and clean rules for re-optimising when workloads and virtual-machine pools change;
- to determine, experimentally and theoretically, the boundary of usefulness of this representation: where list-scheduling heuristics remain better, where dynamic re-optimisation benefits from carried state, and whether "controlled forgetting" ever pays off;
- to demonstrate the approach in a CloudSim-class simulator and in a Kubernetes prototype, so that the contribution is relevant to real cloud resource management;
- to achieve an advanced position in the design and evaluation of nature-inspired optimisation for distributed systems, with a publication record suitable for an academic or industrial research career.

2. The scope of the proposed research

Cloud data centres schedule millions of tasks per day onto heterogeneous virtual machines. Population-based metaheuristics — Particle Swarm Optimisation (PSO), Manta Ray Foraging Optimisation (MRFO), Dwarf Mongoose Optimisation (DMO) and their many relatives — are among the most published approaches to this problem, and a fast-growing subset labels itself "quantum-inspired". In preparatory work (Section 6) I implemented the classical optimisers faithfully and measured, rather than assumed, how they behave on the standard CloudSim-style task-to-VM problem. The result is stark: with the continuous-to-integer rounding encoding that almost all of these papers use, DMO and MRFO perform at random-search level once there are 50–100 tasks, because a single swarm move changes about half of all task assignments. The weakness is the representation, not the metaphor.

The scope of my research is therefore *representation-level* optimisation for cloud scheduling:

- Static scheduling: independent tasks and DAG workflows on heterogeneous VMs, with makespan, energy, cost and deadline objectives, evaluated at equal evaluation budgets against list heuristics (Min-

Min, Max-Min, HEFT), discrete genetic algorithms and probabilistic-model methods (ICPSO, PBIL/EDA), i.e. the *right* baselines rather than only other swarms.

- Dynamic scheduling: task arrivals and departures, VM failures and additions (spot-instance reclamation, scale-out) and performance drift (noisy neighbours), where the optimiser must re-optimize from carried state within a small evaluation budget.
- The quantum-inspired mechanism: one probability-amplitude register per task, Born-rule measurement to obtain schedules, the host swarm's own update equations applied to the registers, best schedules as basis-state attractors, and a depolarising ("decoherence") channel whose strength controls the expected number of reassigned tasks through register purity. The classical twin (probability vectors, linear mixing) is kept throughout so that every claim is ablated.
- Theory: expected-move-size and genetic-drift analysis of register swarms with a noise floor, connecting to the estimation-of-distribution-algorithm theory of margins and drift.
- Systems: CloudSim 7G / WorkflowSim / iFogSim experiments with public traces (GoCJ, HPC2N, Google), and a Kubernetes scheduler-plugin prototype for pod placement under node churn.

Explicitly out of scope: quantum hardware, quantum circuits, quantum annealers and QPU execution. Everything runs on ordinary CPUs with Python/NumPy; the quantum content is a mathematical search mechanism.

3. Overview of the proposed work

The proposed work follows an implementation-first research loop that has already completed one full cycle (observe → hypothesis → implement → run → ablate → adversarial test → prior-art recheck), documented in a runnable notebook, a lab log and a report.

Findings of the completed cycle (5 seeds, 20 000 evaluations per run, four instance families, laptop CPU):

- Failure mode identified: floor-encoded DMO and MRFO change on average 44–49 of 100 task assignments per candidate; fewer than 2 % of candidates improve; gap to the makespan lower bound 76 % (DMO) and 98 % (MRFO) at 100 tasks, versus 1.4 % for a discrete GA at the same budget.
- Mechanism: the measurement-based register representation with a decoherence floor turns MRFO into a GA-class optimiser (1.1 % gap at 100 tasks and 2.8 % at 200 tasks on 20 VMs in a 30-seed, seven-instance confirmation; equal to or slightly better than a discrete GA, Holm-corrected $p < 10^{-4}$ against MRFO on every instance) and repairs DMO likewise (4.0 %; 1.4 % once DMO's unconditional move is made greedy). The decoherence strength shows a clean dose–response with an interior optimum; without it 25–75 % of evaluations are wasted on re-measuring the same schedule. The boundary is measured too: on static heterogeneous batches at this budget the Max-Min list heuristic remains better on six of seven instances by 0.3–4.7 percentage points, which is why the proposal targets objectives and dynamics that list heuristics do not handle.
- Honest negatives: the specifically quantum ingredients (amplitude squaring, signed interference) are measurably irrelevant — a probability-vector twin performs the same — and uniform "decoherence shocks" after a workload change never beat plain continuation at any churn severity from 10 % to 80 %. What does help under change is the representation's structural rules: VM removal as column deletion, VM addition as a uniform share, new tasks as uniform registers — carried state beats restarts on every change type (recovery area 2–4× smaller) and beats the GA's carried population on drift, VM failure and VM addition.

- Prior art: the classical twin is essentially ICPSO (Strasser et al., GECCO 2016) with a new host; the floor is the EDA "margin" / He gate under another name; what is not found anywhere is a purity-calibrated per-candidate floor with a measured dose–response, VM removal as projective column deletion with warm re-optimisation, and any of this in a cloud scheduler.

The PhD builds on this base along three lines:

(a) Representation and theory. Formalise the register swarm as a per-individual univariate probabilistic model driven by swarm difference vectors; derive the expected move size $n - \sum_t \pi_t$ and its evolution under attraction, mixing and the depolarising channel; obtain drift and runtime results (in the style of the chance-constrained makespan analyses of Neumann and co-workers and the r-valued EDA drift results of Doerr and Krejca) that predict the optimal floor strength as a function of n , m and selection pressure; validate the predictions against the measured dose–response.

(b) Re-scheduling under change. Replace uniform forgetting by task-selective, severity-scaled decoherence (shock only the registers of tasks whose execution-time rows changed, scaled by the relative change) and purity-regulated floors; build a dynamic benchmark suite with change generators (arrivals, failures, scale-out, drift) and evaluate against restart, continuation, hypermutation, random immigrants and learning-augmented/online baselines; characterise the severity boundary beyond which carried state stops helping.

(c) Cloud realism. Move from the pure-Python simulator to CloudSim 7G/WorkflowSim/iFogSim with public traces and energy/cost/deadline objectives (where list heuristics no longer dominate), then to a Kubernetes prototype: a scheduler plugin or descheduler that keeps a register state per pending pod set and re-optimises placement under node churn on an iContinuum/GreenK8s-style testbed.

Expected outputs: a first paper on the failure mode of rounding encodings and its repair (workshop/CCGrid-level), a second on severity-aware re-optimisation with a measurement-based representation (journal: FGCS/TPDS-level target), a theory paper on move size and drift with a noise floor (GECCO/PPSN), a systems paper on the Kubernetes prototype, an open-source MEALPY-compatible implementation, and the thesis.

4. Research/Development Methodology

There are two ways to design a "quantum-inspired" optimiser: decorate an existing algorithm with quantum terminology and benchmark it, or start from a measured failure of the classical algorithm, choose a mechanism that changes a specific measurable search property, predict the outcome, and try to falsify it. I follow the second.

- Hypothesis-driven cycle (used in the preparatory work and continued in the PhD): OBSERVE the classical failure with instrumentation (move size, wasted/improving candidates, diversity, purity) → MECHANISM with a stated causal chain → PREDICTION with a falsification condition → MINIMAL IMPLEMENTATION → RUN at equal evaluation budgets with multiple seeds → ABLATION (mechanism removed; classical twin) → ADVERSARIAL cases where the mechanism should not help → PRIOR-ART RECHECK with the exact mechanism → next version. Every version has one evidence-driven change; every number is measured or labelled unmeasured.
- Statistical protocol: 30+ seeds, paired non-parametric tests with Holm correction, effect sizes (Cliff's delta, Vargha–Delaney A12), bootstrap confidence intervals, held-out instance families and real traces; evaluation-budget fairness enforced in code (each measurement is one evaluation).
- Baselines: the algorithms the field uses (floor-encoded swarms), the algorithms it should use (discrete GA, ICPSO, PBIL/EDA, list heuristics, HEFT), and for dynamic settings restart/continue/hypermutation/random immigrants and learning-augmented online baselines.

- Theory: expected-move-size calculus of registers; drift analysis with margins; runtime bounds for the register swarm on makespan scheduling under chance constraints.
- Systems: CloudSim 7G/WorkflowSim/iFogSim (the laboratory-standard toolkits) with GoCJ/HPC2N/Google traces; Kubernetes prototype with failure injection; Docker-reproducible environments (the preparatory work already ships a Dockerfile and a notebook that runs end-to-end in smoke/fast/full modes).
- Reproducibility: all code, instances, seeds, logs and literature maps are released; the preparatory package includes 18 automated validation tests (objective versus brute force, Born-rule measurement chi-square test, budget accounting, determinism).

5. The expected outcomes of the research

- A demonstrated, reproducible failure-mode analysis showing why rounding-encoded swarm schedulers degrade with problem size — a result that affects a large published literature and that the field currently does not measure.
- A measurement-based register representation with a purity-calibrated noise floor that makes swarm optimisers competitive with discrete GAs at equal budgets, with a theory that predicts the floor strength.
- A characterisation of re-scheduling under change: which changes benefit from carried state, how the benefit decays with severity, and whether selective decoherence has a regime of usefulness — together with a dynamic cloud-scheduling benchmark suite.
- A negative-result contribution that matters for the community: quantum-specific ingredients (Born rule, interference) and uniform forgetting add nothing measurable on this problem class, which sets a standard of evidence for "quantum-inspired" schedulers.
- Simulator-validated and Kubernetes-prototyped resource management under churn, with energy-aware objectives, contributing to the knowledge of sustainable, adaptive cloud resource management.
- Open-source software (MEALPY-compatible optimisers, CloudSim modules, the Kubernetes plugin) and four to five peer-reviewed publications.

6. Reference to any preparatory reading and work that has been completed

- Implementation-first pilot (2026): a runnable Jupyter notebook (`quantum_inspired_cloud_scheduler.ipynb`, 18 sections: problem model, faithful PSO/DMO/MRFO/GA implementations, the quantum-inspired mechanism, validation tests, baseline, convergence, statistics, ablation, sensitivity, adversarial cases, dynamic workloads, interpretation) executed end-to-end in Docker; a lab log with the full version history (V0–V3 and the dynamic pilots); a research report; live-web literature maps of quantum-inspired PSO (23 papers), DMO (5 quantum variants, 9 cloud applications), MRFO (6 quantum items, 12 cloud applications), quantum-inspired cloud scheduling on other hosts (19 papers) and the classical equivalents of every quantum mechanism (rotation gate = PBIL, tunnelling = Lévy/Cauchy mutation, decoherence = mutation shift / hypermutation, superposition + measurement = univariate EDA, purity floor = EDA margins); and a prior-art recheck of the exact implemented mechanism (ICPSO, QIEA/He gate, PBIL hypermutation, ACO pheromone equalisation, qudit-inspired optimisation).
- B.Tech minor project (PDEU, 2025–2026): Dwarf Mongoose Optimisation applied to cloud VM scheduling, benchmarked against ACO and PSO — the starting point from which the present, more critical measurement of DMO's behaviour grew.

- Professional experience directly relevant to the systems component: production AWS infrastructure with Terraform (multi-AZ VPC, ALB/ASG, RDS, CodePipeline), Docker/Kubernetes deployments with zero-downtime rollouts and automated rollback, CloudWatch cost/utilisation dashboards, and a Lambda/EventBridge scheduler for cost control; Kubernetes (CKA-track) and Terraform Associate training; three DevOps internships (Workability Tech, MGrow, Growexx) and a software-development internship (L&T).
- Reading completed: Zhao, Zhang & Wang (2020) MRFO; Agushaka, Ezugwu & Abualigah (2022) DMO and the MATLAB/MEALPY discrepancies; Sun et al. (2004–2012) QPSO and its bare-bones/Laplace reading (Mikki & Kishk 2006); Han & Kim (2002, 2004) QIEA and the H_e gate; Platel, Schliebs & Kasabov (2009) QEA as an EDA; Baluja (1994) PBIL; Krejca & Witt (2018) EDA theory; Strasser et al. (2016) ICPSO; Yang & Yao (2005/2008), Yang & Richter (2009) PBIL in dynamic environments; Guntsch & Middendorf (2001) pheromone equalisation; Neumann and co-workers on chance-constrained makespan scheduling (TCS 2025) and diversity optimisation; the 2025 MRFO survey (Yahia et al.) and the 2024 MRFO survey (Gharehchopogh et al.); the qCLOUDS Lab 2025 Annual Report, iQuantum/CloudSim 7G, GreenK8s (Toosi et al. 2025), REACH (Bai, Islam, Buyya, Toosi 2025), and the learning-augmented scheduling line of Zomaya and co-workers (ICLR 2025).

7. Fit with the host laboratory and scholarship alignment

The proposed research sits at the intersection of the CLOUDS/qCLOUDS Laboratory's core probes — cloud workflows and scheduling, energy-efficient clouds, the CloudSim toolkit, fog/edge computing and AI for next-generation clouds — and the current systems work of the DisNet Laboratory on Kubernetes scheduling (GreenK8s), resilience under failure injection and adaptive microservice rescheduling (REACH, ADASCALE, TraDE). It uses the laboratory's own toolkit (CloudSim 7G) for evaluation and contributes a scheduling method with an explicit theory of when it helps, evaluated against DRL and list-scheduling baselines the laboratory already publishes with. The theoretical component aligns with the evolutionary-computation-under-dynamic-constraints programme at Adelaide (Neumann) and can be structured as a co-supervised or collaborative strand.

Alignment with Research Training Program (RTP) selection criteria: the proposal is grounded in a completed, reproducible research artefact (notebook, report, code, literature maps) rather than in intentions; it states falsifiable hypotheses, reports negative results, and commits to a statistical protocol — evidence of research potential and of the capacity to complete a doctorate within the funded period. The candidate's academic record (CPI 8.84/10, top-decile engineering programme), production cloud-engineering experience (AWS, Terraform, Docker, Kubernetes, CI/CD) and the systems-level component of the plan (CloudSim and Kubernetes prototype) match the applied, high-impact orientation of the laboratories. The candidate will pursue a peer-reviewed publication from the preparatory work before the application round, since RTP rankings at Melbourne, Monash, Sydney and Adelaide weigh peer-reviewed outputs and prior research experience alongside grades.

8. Timeline (24 months, with milestones)

- Months 0–3: literature and fundamentals; CloudSim 7G/WorkflowSim setup; formal statement of the register-swarm model. Milestone: candidature proposal and workshop paper submitted (failure-mode analysis and repair).
- Months 3–6: full study on held-out instances and public traces; Max-Min-seeded and energy/cost/deadline objectives. Milestone: static-scheduling results paper.
- Months 6–10: selective, severity-scaled decoherence; purity-regulated floor; dynamic benchmark suite and change generators. Milestone: dynamic re-optimisation paper.
- Months 10–14: theory of expected move size, drift and runtime with a noise floor; validation against the dose–response. Milestone: theory paper (GECCO/PPSN).

- Months 14–18: large-scale CloudSim/WorkflowSim experiments with DAG workflows, 30+ seeds, DRL and list baselines. Milestone: journal paper.
- Months 18–21: Kubernetes scheduler-plugin prototype with failure injection on a cluster testbed. Milestone: systems paper and open-source release.
- Months 21–24: thesis writing and submission.

9. Selected references

Agushaka, J.O., Ezugwu, A.E., Abualigah, L. (2022). Dwarf mongoose optimization algorithm. CMAME 391:114570. · Zhao, W., Zhang, Z., Wang, L. (2020). Manta ray foraging optimization. EAAI 87:103300. · Sun, J., Fang, W., Palade, V., Wu, X., Xu, W. (2011). Quantum-behaved PSO with Gaussian distributed local attractor point. AMC 218:3763. · Han, K.-H., Kim, J.-H. (2002, 2004). Quantum-inspired evolutionary algorithms. IEEE TEVC 6(6); 8(2). · Platel, M.D., Schliebs, S., Kasabov, N. (2009). Quantum-inspired evolutionary algorithm: a multimodel EDA. IEEE TEVC 13(6). · Baluja, S. (1994). Population-based incremental learning. CMU-CS-94-163. · Krejca, M., Witt, C. (2018). Theory of estimation-of-distribution algorithms. arXiv 1806.05392. · Strasser, S., Goodman, R., Sheppard, J., Butcher, S. (2016). A new discrete particle swarm optimization algorithm. GECCO 2016. · Yang, S., Richter, H. (2009). Hyper-learning for PBIL in dynamic environments. IEEE CEC 2009. · Guntsch, M., Middendorf, M. (2001). Pheromone modification strategies for ant algorithms applied to dynamic TSP. EvoWorkshops 2001. · Balicki, J. (2022). Many-objective quantum-inspired PSO for VM placement. Entropy 24(1):58. · Yahia et al. (2025). A comprehensive survey of MRFO. Arch. Comput. Methods Eng. · Neumann, F. et al. (2025). Runtime performance of EAs for the chance-constrained makespan scheduling problem. TCS. · Sun, Y., Xu, M., Toosi, A.N. (2025). GreenK8s: green-aware scheduling for sustainable Kubernetes cluster management. IEEE CLUSTER 2025. · Bai, Islam, Buyya, Toosi (2025). REACH: RL for adaptive microservice rescheduling in the cloud-edge continuum. arXiv 2510.06675. · Buyya, R. et al. (2025). qCLOUDS Laboratory Annual Report 2025; iQuantum; CloudSim 7G. · Wu, Li, Wang, Zomaya (2018). Multi-objective EDA for fog scheduling. IEEE TCC.

Enclosures: quantum_inspired_research_report.md (PDF), quantum_inspired_cloud_scheduler.ipynb, README.md, lab_log.md, literature/.